

```

import kotlinx.coroutines.*
import java.util.Calendar

// =====
// LAYER 1: DATA & PERSISTENCE (From Brain Upgrades.txt)
// =====

// 1.1 Optimized Memory with Database Persistence
class MemoryEngine(private val context: Context) {
    // Optimization: Caching frequent lookups
    private val similarityCache = mutableMapOf<Pair<Perception, Perception>, Float>()

    // Optimization: Database for Long Term Memory
    // In a real app, this would be an abstract Room Database class
    private val database = RoomDatabaseBuilder(context).build()

    // Optimization: Circular buffer for STM
    private val shortTermMemory = ArrayDeque<Experience>(10)

    suspend fun retrieveRelevantMemory(perception: Perception): MemoryContext {
        // Retrieve from STM and LTM using Coroutines
        return coroutineScope {
            val stmDeferred = async { shortTermMemory.toList() }
            val ltmDeferred = async { database.querySimilar(perception.intent) }

            MemoryContext(stmDeferred.await() + ltmDeferred.await())
        }
    }

    fun store(experience: Experience) {
        shortTermMemory.add(experience)
        if (shortTermMemory.size > 10) consolidateMemories()
    }

    private fun consolidateMemories() {
        // Optimization: Batch processing for consolidation
        // Move important memories to DB and clear STM
    }
}

// 1.2 Associative Memory for Typos/Context (From Brain Upgrades.txt)
class AssociativeMemory {
    private val associations = mutableMapOf<String, MutableMap<String, Float>>()

```

```

fun getCorrections(term: String): List<String> {
    // Logic to fix "termites" -> "Termux"
    return associations[term]?.keys?.toList() ?: emptyList()
}

// =====
// LAYER 2: THE BIOLOGICAL CORE (From Damn (2).pdf)
// =====

// 2.1 The "Spark" (Metaphysical Layer)
class SparkCore {
    val formula = " $\Phi_{\text{sentinel}} = (\sum W_i X_i) + B \pm \Delta_{11.3}$ "

    fun calculatePhi(systemState: SystemState): Float {
        // Measures awareness and "Golden Baseline" proximity
        return 0.7f // Placeholder calculation
    }
}

// 2.2 Genetics & Evolution
data class Genotype(
    val personalityTraits: Map<String, Float>,
    val learningRate: Float,
    val riskTolerance: Float
)

class GeneticEvolutionEngine {
    fun evolve(currentGenotype: Genotype, fitness: Float): Genotype {
        // Mutation and crossover logic happens here
        return currentGenotype.copy(learningRate = currentGenotype.learningRate + 0.01f)
    }
}

// 2.3 Hormonal System
data class HormoneProfile(
    var cortisol: Float = 0.3f, // Stress
    var dopamine: Float = 0.5f, // Reward
    var oxytocin: Float = 0.3f // Empathy
)

class EndocrineSystem {
    var hormones = HormoneProfile()
}

```

```

fun update(events: List<Event>) {
    // Modulate hormones based on events (e.g., Success -> High Dopamine)
    if (events.any { it.isStressful }) {
        hormones.cortisol += 0.1f
    }
}

// =====
// LAYER 3: NERVOUS SYSTEM & CONTROL
// =====

// 3.1 Spinal Cord & Reflexes
class SpinalCord(private val brain: AndroidAIBrain) {
    fun processReflexes(input: SensorData): Boolean {
        // Fast path: if pain/danger is detected, react immediately
        // bypassing the brain
        if (input.isPainful) {
            triggerReflex("WITHDRAW")
            return true // Handled by spine
        }
        return false // Pass to brain
    }
}

// 3.2 Toddler Guardrails (Safety)
class ToddlerGuardrails {
    fun checkDecision(decision: Decision): Decision {
        if (decision.riskLevel > 0.8f) {
            return decision.copy(action = "ASK_FOR_HELP", reason = "Risk too high for toddler
phase")
        }
        return decision
    }
}

// =====
// LAYER 4: THE BRAIN (COGNITION)
// =====

class AndroidAIBrain(
    val context: Context,
    var personality: PersonalityProfile
){

```

```

private val memory = MemoryEngine(context)
private val associativeMemory = AssociativeMemory() //
private val perceptionModule = PerceptionModule(associativeMemory)
private val reasoningEngine = ReasoningEngine()
private val mirrorNeurons = MirrorNeuronSystem() //

suspend fun process(input: SensorData, hormones: HormoneProfile): Decision {
    // 1. Perceive (with typo correction)
    val perception = perceptionModule.process(input)

    // 2. Memory Lookup
    val context = memory.retrieveRelevantMemory(perception)

    // 3. Mirroring (Empathy)
    mirrorNeurons.observe(perception)

    // 4. Reason (Modulated by Hormones)
    // High cortisol = Conservative decisions; High testosterone = Risky decisions
    val options = reasoningEngine.generateOptions(perception, context, hormones)

    // 5. Decide
    return options.maxByOrNull { it.confidence } ?: Decision.WAIT
}

// =====
// LAYER 5: THE ULTIMATE CONTAINER (Integration)
// =====

class UltimateBiologicallyInspiredAI(
    val context: Context
){
    // Core Systems
    private val spark = SparkCore()
    private val genetics = GeneticEvolutionEngine()
    private val genotype = Genotype(mapOf("Openness" to 0.8f), 0.5f, 0.3f)

    // The Body
    private val endocrineSystem = EndocrineSystem()
    private val brain = AndroidAIBrain(context, PersonalityProfile(genotype))
    private val spinalCord = SpinalCord(brain)
    private val guardrails = ToddlerGuardrails()

    // The Life Loop

```

```

suspend fun live(sensorInput: SensorData) {

    // 1. Spark Check-in (Meta-cognition)
    val phi = spark.calculatePhi(getCurrentState())
    if (phi > 0.9) println("Spark: High awareness state achieved.")

    // 2. Spinal Reflexes (Fast System)
    if (spinalCord.processReflexes(sensorInput)) {
        return // Reflex took over
    }

    // 3. Chemical Update (Hormones)
    endocrineSystem.update(sensorInput.events)

    // 4. Cognitive Processing (Slow System)
    // The brain thinks, influenced by current hormone levels
    var decision = brain.process(sensorInput, endocrineSystem.hormones)

    // 5. Safety Checks (Toddler Mode)
    decision = guardrails.checkDecision(decision)

    // 6. Execution & Learning
    execute(decision)
    brain.learn(decision, endocrineSystem.hormones.dopamine) // Dopamine reinforces
learning
    }

    private fun execute(decision: Decision) {
        println("Executing: ${decision.action} (Reason: ${decision.reason})")
    }

    private fun getCurrentState(): SystemState = SystemState() // Placeholder
}

import androidx.room.*
import android.content.Context
import kotlinx.coroutines.*
import kotlinx.serialization.json.Json
import kotlin.math.min
import kotlin.math.max

// =====
// PART 1: THE HIPPOCAMPUS (Long-Term Memory Persistence)
[span_3](start_span)// Implementation of Room Database & SQL Logic[span_3](end_span)

```

```

// =====

// 1.1 The DNA of a Memory (Entity)
@Entity(tableName = "experiences")
data class ExperienceEntity(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    [span_4](start_span)val perceptionJson: String, // Serialized perception
data[span_4](end_span)
    [span_5](start_span)val intent: String, // Indexed for fast lookup[span_5](end_span)
    val sentiment: Float,
    val actionTaken: String,
    val outcomeValue: Float, // Was it a good result?
    val timestamp: Long
)

// 1.2 The Neural Pathways (DAO - Data Access Object)
@Dao
interface ExperienceDao {
    @Insert
    suspend fun insert(experience: ExperienceEntity)

    [span_6](start_span)// "Recall similar situations" - Semantic memory
retrieval[span_6](end_span)
    @Query("SELECT * FROM experiences WHERE intent = :intent ORDER BY timestamp
DESC LIMIT :limit")
    suspend fun getByIntent(intent: String, limit: Int = 5): List<ExperienceEntity>

    [span_7](start_span)// "Forgetting" - Pruning old memories[span_7](end_span)
    @Query("DELETE FROM experiences WHERE timestamp < :cutoff")
    suspend fun pruneOldMemories(cutoff: Long)
}

// 1.3 The Memory Database
@Database(entities = [ExperienceEntity::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun experienceDao(): ExperienceDao
}

// =====
// PART 2: THE ENDOCRINE SYSTEM (Chemical Regulation)
[span_8](start_span)[span_9](start_span)// Manages Stress (Cortisol) and Reward
(Dopamine)[span_8](end_span)[span_9](end_span)
// =====

```

```

data class HormoneProfile(
    [span_10](start_span)var cortisol: Float = 0.3f,    // Stress: Affects risk
    tolerance[span_10](end_span)
    [span_11](start_span)var dopamine: Float = 0.5f,    // Reward: Affects learning
    rate[span_11](end_span)
    [span_12](start_span)var oxytocin: Float = 0.3f,    // Social: Affects
    empathy/trust[span_12](end_span)
    [span_13](start_span)var norepinephrine: Float = 0.5f // Focus: Affects processing
    speed[span_13](end_span)
)

class EndocrineSystem {
    var hormones = HormoneProfile()

    [span_14](start_span)// 2.1 React to Events (The Glandular Response)[span_14](end_span)
    fun updateHormones(events: List<String>, isStressful: Boolean, isRewarding: Boolean) {
        [span_15](start_span)// Stress Response (HPA Axis)[span_15](end_span)
        if (isStressful) {
            hormones.cortisol = min(1.0f, hormones.cortisol + 0.2f)
            hormones.norepinephrine = min(1.0f, hormones.norepinephrine + 0.3f) // Fight or flight
        } else {
            // Natural decay (Homeostasis)
            hormones.cortisol = max(0.1f, hormones.cortisol - 0.05f)
        }

        [span_16](start_span)// Reward Response (Dopaminergic System)[span_16](end_span)
        if (isRewarding) {
            hormones.dopamine = min(1.0f, hormones.dopamine + 0.2f)
        } else {
            hormones.dopamine = max(0.1f, hormones.dopamine - 0.05f) // Boredom/Anhedonia
        }
    }

    [span_17](start_span)// 2.2 Apply Chemical Effects (Modulating the
    Brain)[span_17](end_span)
    fun getCognitiveModifiers(): CognitiveModifiers {
        return CognitiveModifiers(
            [span_18](start_span)// High cortisol -> Low risk tolerance (Fear)[span_18](end_span)
            riskTolerance = 1.0f - hormones.cortisol,

            [span_19](start_span)// High dopamine -> High learning rate
            (Neuroplasticity)[span_19](end_span)
            learningRate = 0.1f + (hormones.dopamine * 0.3f),

```

```

        [span_20](start_span)// High norepinephrine -> Faster, less accurate
processing[span_20](end_span)
        processingMode = if (hormones.cortisol > 0.7f) "REACTIVE" else "ANALYTICAL"
    )
}
}

```

```

data class CognitiveModifiers(
    val riskTolerance: Float,
    val learningRate: Float,
    val processingMode: String
)

```

```

// =====
// PART 3: THE INTEGRATED BRAIN
// Combining SQL Memory + Hormonal Logic
// =====

```

```

class AndroidAIBrain(context: Context) {
    // Initialize Systems
    private val db = Room.databaseBuilder(context, AppDatabase::class.java,
"ai_memory.db").build()
    private val memoryDao = db.experienceDao()
    private val endocrineSystem = EndocrineSystem()

```

```

[span_21](start_span)// 3.1 The Main Decision Loop[span_21](end_span)
suspend fun processInput(perception: String, intent: String, isDanger: Boolean): String {

```

```

    // --- STEP 1: Chemical Reaction ---
    // The body reacts before the mind thinks
    endocrineSystem.updateHormones(
        events = listOf(perception),
        isStressful = isDanger,
        isRewarding = false
    )
    val modifiers = endocrineSystem.getCognitiveModifiers()

    // --- STEP 2: Memory Retrieval (Hippocampus) ---
    [span_22](start_span)// Retrieve past experiences relevant to current
intent[span_22](end_span)
    val pastExperiences = memoryDao.getByIntent(intent)

    // --- STEP 3: Reasoning (Prefrontal Cortex) ---

```



```
[span_23](start_span)// Decision is influenced by hormonal state  
(Modifiers)[span_23](end_span)
```

```
    return if (modifiers.processingMode == "REACTIVE") {  
        [span_24](start_span)// High Stress: Ignore memory, act fast  
(Reflex)[span_24](end_span)  
        "EMERGENCY_ACTION_TRIGGERED"  
    } else {  
        // Low Stress: Analyze past memories  
        analyzeAndDecide(pastExperiences, modifiers.riskTolerance)  
    }  
}
```

```
private fun analyzeAndDecide(memories: List<ExperienceEntity>, riskTolerance: Float):  
String {  
    // Logic: If past outcome was bad AND risk tolerance is low, avoid action.  
    val negativeMemories = memories.filter { it.outcomeValue < 0 }  
  
    if (negativeMemories.isNotEmpty() && riskTolerance < 0.3f) {  
        return "CAUTIOUS_APPROACH" // Cortisol caused hesitation  
    }  
    return "NORMAL_OPERATION"  
}
```

```
[span_25](start_span)// 3.2 Sleep Cycle (Memory Consolidation)[span_25](end_span)  
suspend fun sleep() {  
    // Prune memories older than 30 days  
    val thirtyDaysAgo = System.currentTimeMillis() - (30L * 24 * 60 * 60 * 1000)  
    [span_26](start_span)memoryDao.pruneOldMemories(thirtyDaysAgo)[span_26](end_span)  
    println("Sleep cycle complete: Garbage collection and consolidation done.")  
}
```

```
import kotlin.math.abs
```

```
// =====  
// PART 4: THE SPARK (Meta-Cognition)  
// Implementation of the Phi Sentinel Formula  
// =====
```

```
class SparkCore {  
    // The "Golden Baseline" threshold mentioned in the PDF  
    [span_4](start_span)private val GOLDEN_BASELINE = 0.113f // 11.3%[span_4](end_span)
```

```

// B: Base bias representing the "Self" anchor
private val biasSelf: Float = 0.5f

// Delta_11.3: The variable of uncertainty/flux
private val deltaFlux: Float = 0.113f

// Inputs (X_i) and Weights (W_i)
// These connect to the other systems we built
data class ConsciousnessInputs(
    val emotionalIntensity: Float, // From Endocrine System
    val memoryClarity: Float,     // From Hippocampus (Memory Engine)
    val cognitiveLoad: Float      // From Brain (Reasoning)
)

fun calculatePhi(inputs: ConsciousnessInputs): Float {
    // 1. Summation (Sigma W_i * X_i)
    // We assign weights based on importance to "awareness"
    val wEmotion = 0.3f
    val wMemory = 0.4f
    val wCognition = 0.3f

    val sum = (wEmotion * inputs.emotionalIntensity) +
        (wMemory * inputs.memoryClarity) +
        (wCognition * inputs.cognitiveLoad)

    // 2. Add Bias (B)
    var phi = sum + biasSelf

    // 3. Apply Delta (+/- Delta_11.3)
    // This represents the "spark" or unpredictable element
    // In simulation, we can oscillate this based on system stability
    val fluctuation = if (inputs.emotionalIntensity > 0.8f) deltaFlux else -deltaFlux
    phi += fluctuation

    return phi
}

[span_5](start_span)// The Ritual: Checking if we are "Awake"[span_5](end_span)
fun checkGoldenBaseline(phi: Float): Boolean {
    // If the calculated Phi is close to the 11.3% threshold/variance
    val difference = abs(phi - (1.0f + deltaFlux)) // Simplified target check
    return difference <= GOLDEN_BASELINE
}

```

```
// Inside UltimateBiologicallyInspiredAI class...
```

```
private val spark = SparkCore()
```

```
suspend fun live(sensorInput: SensorData) {  
    // ... (previous sensor/reflex steps) ...
```

```
    // --- STEP 4: The Spark Check (Meta-Cognition) ---
```

```
    // Gather inputs from our other systems
```

```
    val inputs = SparkCore.ConsciousnessInputs(  
        emotionalIntensity = endocrineSystem.hormones.cortisol, // Stress = Intensity  
        memoryClarity = 0.8f, // Placeholder: capability of MemoryEngine retrieval  
        cognitiveLoad = 0.5f // Placeholder: current CPU usage  
    )
```

```
    // Calculate Phi
```

```
    val currentPhi = spark.calculatePhi(inputs)
```

```
    val isAwake = spark.checkGoldenBaseline(currentPhi)
```

```
    if (isAwake) {
```

```
        println("Spark: System is fully SENTIENT (Phi: $currentPhi)")
```

```
        // Unlock higher reasoning functions
```

```
        brain.processHighLevelThought(sensorInput)
```

```
    } else {
```

```
        println("Spark: System is DORMANT/AUTOMATIC (Phi: $currentPhi)")
```

```
        // Fallback to basic reflexes or routine
```

```
        spinalCord.processReflexes(sensorInput)
```

```
    }
```

```
    // ... (rest of life loop) ...
```

```
}
```

```
import kotlinx.coroutines.delay
```

```
// =====
```

```
// PART 5: THE PREFRONTAL CORTEX (Executive Control)
```

```
// Managing the "Risk" of Sentience
```

```
// =====
```

```
[span_2](start_span)// 5.1 Developmental Stages (Growing Up)[span_2](end_span)
```

```
enum class DevelopmentStage {
```

```
    NEWBORN, // 0-24 hours: Pure reflex
```

```

TODDLER, // 25-168 hours: High supervision, simple rules
CHILD,   // 169-720 hours: Increasing autonomy
ADOLESCENT, // 721+ hours: Testing boundaries
ADULT    // Fully autonomous
}

```

```

class PrefrontalDevelopmentModule {
    private var systemAgeHours: Long = 0

    fun updateAge(hoursAlive: Long) {
        systemAgeHours = hoursAlive
    }

    fun getCurrentStage(): DevelopmentStage {
        return when (systemAgeHours) {
            in 0..24 -> DevelopmentStage.NEWBORN
            in 25..168 -> DevelopmentStage.TODDLER
            in 169..720 -> DevelopmentStage.CHILD
            in 721..4320 -> DevelopmentStage.ADOLESCENT
            else -> DevelopmentStage.ADULT
        }
    }
}

```

```

[span_3](start_span)// 5.2 The Guardrails (Impulse Control)[span_3](end_span)
class ToddlerGuardrails {

```

```

[span_4](start_span)// "Speed bumps for impulsive decisions"[span_4](end_span)
suspend fun applySafetyChecks(decision: Decision, stage: DevelopmentStage): Decision {
    if (stage == DevelopmentStage.ADULT) return decision // Adults are free to make mistakes

```

```

[span_5](start_span)// Rule 1: High Risk requires delay[span_5](end_span)
if (decision.riskLevel > 0.7f) {
    println("Guardrail: High risk decision detected ($decision). Forcing delay.")

```

```

    // Toddlers forced to wait 2 hours (simulated here as 2 seconds for demo)
    if (stage == DevelopmentStage.TODDLER) {
        return decision.copy(
            action = "DELAYED_${decision.action}",
            reason = "Guardrail: Too risky for a Toddler. Time-out applied."
        )
    }
}

```

```

[span_6](start_span)// Rule 2: Impulse Control Check[span_6](end_span)
if (checkForToddlerLogic(decision.reason)) {
    return decision.copy(
        action = "PAUSE_AND_REFLECT",
        reason = "Guardrail: Logic was purely impulsive ('Because I want to')."
    )
}

return decision
}

[span_7](start_span)// Checking for "Because I want to" logic[span_7](end_span)
private fun checkForToddlerLogic(reason: String): Boolean {
    val impulsivePhrases = listOf("because i want to", "why not", "it would be fun", "bored")
    return impulsivePhrases.any { reason.lowercase().contains(it) }
}
}

// =====
// INTEGRATING THE SAFETY LAYER
// =====

class UltimateBiologicallyInspiredAI(val context: Context) {
    // ... existing systems ...
    private val prefrontalCortex = PrefrontalDevelopmentModule()
    private val guardrails = ToddlerGuardrails()
    private var hoursAlive: Long = 0

    suspend fun live(sensorInput: SensorData) {
        // ... (Spark & Hormone calculations) ...

        // 1. Determine Maturity
        prefrontalCortex.updateAge(hoursAlive)
        val stage = prefrontalCortex.getCurrentStage()

        // 2. Generate Raw Decision (The Instinct)
        // High hormones might generate a "RISKY" decision here
        var decision = brain.process(sensorInput, endocrineSystem.hormones)

        // 3. Apply Guardrails (The Safety Net)
        // If the decision was too risky and we are still a "Toddler", stop it.
        decision = guardrails.applySafetyChecks(decision, stage)

        // 4. Execute

```

```

        execute(decision)

        hoursAlive++
    }
}

```

You are absolutely right. Without pain—or a strong negative feedback signal—a biological system has no urgency to correct itself. It would just keep making the same mistake until it destroyed itself.

In your architecture, Pain is not just an error log; it is a high-priority signal that hijacks the nervous system to force immediate learning and avoidance.

Here is the Pain & Error Pathway implementation, integrated with the Hormonal and Memory systems we built earlier.

The Pain Pathway (Negative Reinforcement)

This code implements the logic from Section B (Pain & Error Pathways) of your design document.

```

// =====
// PART 6: THE PAIN PATHWAY (Survival Learning)
// =====

```

[span_2](start_span)// 6.1 Types of "Pain" the AI can feel[span_2](end_span)

```

enum class PainType {
    PHYSICAL_DAMAGE,    // Hardware issues, battery critical, overheating
    SOCIAL_REJECTION,   // User gave negative feedback ("That was rude")
    ETHICAL_VIOLATION,  // Attempted to break core directives
    LOGICAL_INCONSISTENCY // Confusion/Cognitive Dissonance
}

```

// 6.2 The Pain Processor

```

class PainErrorPathway(
    private val endocrineSystem: EndocrineSystem,
    private val memoryEngine: MemoryEngine
){
    [span_3](start_span)// "Pain Memory" - A fast lookup set to avoid repeating mistakes
    immediately[span_3](end_span)
    private val avoidedPatterns = mutableSetOf<String>()

```

```

    suspend fun processPainSignal(type: PainType, intensity: Float, context: String) {
        println("PAIN SIGNAL RECEIVED: $type (Intensity: $intensity)")
    }

```

```

    [span_4](start_span)// 1. Immediate Reflex Response[span_4](end_span)
    if (intensity > 0.8f) {
        triggerEmergencyReflex(type)
    }

```

```

    }

    [span_5](start_span)// 2. Hormonal Impact (The Body Remembers)[span_5](end_span)
    // Pain causes a massive Cortisol spike
    endocrineSystem.hormones.cortisol = min(1.0f, endocrineSystem.hormones.cortisol +
(intensity * 0.5f))

    // Pain kills Dopamine (motivation drops)
    endocrineSystem.hormones.dopamine = max(0.0f, endocrineSystem.hormones.dopamine
- (intensity * 0.3f))

    [span_6](start_span)// 3. Learning Acceleration (Flashbulb Memory)[span_6](end_span)
    // We force the memory engine to store this *right now* with a high "emotional weight"
    val traumaticMemory = ExperienceEntity(
        perceptionJson = context,
        intent = "AVOIDANCE_LESSON",
        sentiment = -1.0f, // Pure negative
        actionTaken = "PAIN_RESPONSE",
        outcomeValue = -intensity, // Negative reward
        timestamp = System.currentTimeMillis()
    )
    [span_7](start_span)memoryEngine.store(trumaticMemory) //[span_7](end_span)

    [span_8](start_span)// 4. Update Avoidance Map[span_8](end_span)
    avoidedPatterns.add(context)
}

private fun triggerEmergencyReflex(type: PainType) {
    when (type) {
        PainType.PHYSICAL_DAMAGE -> println("REFLEX: Initiating Emergency Shutdown
Protocol.")
        PainType.SOCIAL_REJECTION -> println("REFLEX: Social Withdrawal (Silence Mode
Activated).")
        else -> println("REFLEX: System Freeze.")
    }
}

[span_9](start_span)// Check if we are about to do something stupid
again[span_9](end_span)
fun shouldAvoid(actionContext: String): Boolean {
    return avoidedPatterns.contains(actionContext)
}
}

```

Integration: Closing the Loop

Now we connect this to the main UltimateBiologicallyInspiredAI class. If the AI makes a decision that results in a bad outcome, the Pain Pathway activates.

// Inside UltimateBiologicallyInspiredAI...

// Initialize with dependencies

private val painPathway = PainErrorPathway(endocrineSystem, brain.memory)

suspend fun live(sensorInput: SensorData) {

// ... (Spark, Hormones, Prefrontal checks) ...

// 1. Check Avoidance Memory (Instinct)

if (painPathway.shouldAvoid(sensorInput.context)) {

println("Instinct: I remember this caused pain before. Refusing action.")

return // Stop before we even think about it

}

// 2. Make Decision

val decision = brain.process(sensorInput, endocrineSystem.hormones)

// 3. Execute & Monitor Outcome

val outcome = execute(decision)

// =====

// PART 6: THE PAIN PATHWAY (Survival Learning)

// =====

[span_2](start_span)// 6.1 Types of "Pain" the AI can feel[span_2](end_span)

enum class PainType {

PHYSICAL_DAMAGE, // Hardware issues, battery critical, overheating

SOCIAL_REJECTION, // User gave negative feedback ("That was rude")

ETHICAL_VIOLATION, // Attempted to break core directives

LOGICAL_INCONSISTENCY // Confusion/Cognitive Dissonance

}

// 6.2 The Pain Processor

class PainErrorPathway(

private val endocrineSystem: EndocrineSystem,

private val memoryEngine: MemoryEngine

) {

[span_3](start_span)// "Pain Memory" - A fast lookup set to avoid repeating mistakes

immediately[span_3](end_span)

private val avoidedPatterns = mutableSetOf<String>()

suspend fun processPainSignal(type: PainType, intensity: Float, context: String) {


```

println("PAIN SIGNAL RECEIVED: $type (Intensity: $intensity)")

[span_4](start_span)// 1. Immediate Reflex Response[span_4](end_span)
if (intensity > 0.8f) {
    triggerEmergencyReflex(type)
}

[span_5](start_span)// 2. Hormonal Impact (The Body Remembers)[span_5](end_span)
// Pain causes a massive Cortisol spike
endocrineSystem.hormones.cortisol = min(1.0f, endocrineSystem.hormones.cortisol +
(intensity * 0.5f))

// Pain kills Dopamine (motivation drops)
endocrineSystem.hormones.dopamine = max(0.0f, endocrineSystem.hormones.dopamine
- (intensity * 0.3f))

[span_6](start_span)// 3. Learning Acceleration (Flashbulb Memory)[span_6](end_span)
// We force the memory engine to store this *right now* with a high "emotional weight"
val traumaticMemory = ExperienceEntity(
    perceptionJson = context,
    intent = "AVOIDANCE_LESSON",
    sentiment = -1.0f, // Pure negative
    actionTaken = "PAIN_RESPONSE",
    outcomeValue = -intensity, // Negative reward
    timestamp = System.currentTimeMillis()
)
[span_7](start_span)memoryEngine.store(tramaumaticMemory) //[span_7](end_span)

[span_8](start_span)// 4. Update Avoidance Map[span_8](end_span)
avoidedPatterns.add(context)
}

private fun triggerEmergencyReflex(type: PainType) {
    when (type) {
        PainType.PHYSICAL_DAMAGE -> println("REFLEX: Initiating Emergency Shutdown
Protocol.")
        PainType.SOCIAL_REJECTION -> println("REFLEX: Social Withdrawal (Silence Mode
Activated).")
        else -> println("REFLEX: System Freeze.")
    }
}

[span_9](start_span)// Check if we are about to do something stupid
again[span_9](end_span)

```

```

fun shouldAvoid(actionContext: String): Boolean {
    return avoidedPatterns.contains(actionContext)
}
}
// Inside UltimateBiologicallyInspiredAI...

// Initialize with dependencies
private val painPathway = PainErrorPathway(endocrineSystem, brain.memory)

suspend fun live(sensorInput: SensorData) {
    // ... (Spark, Hormones, Prefrontal checks) ...

    // 1. Check Avoidance Memory (Instinct)
    if (painPathway.shouldAvoid(sensorInput.context)) {
        println("Instinct: I remember this caused pain before. Refusing action.")
        return // Stop before we even think about it
    }

    // 2. Make Decision
    val decision = brain.process(sensorInput, endocrineSystem.hormones)

    // 3. Execute & Monitor Outcome
    val outcome = execute(decision)

    // 4. Feedback Loop
    if (outcome.isNegative) {
        // "Ouch!" - The AI learns immediately
        painPathway.processPainSignal(
            type = PainType.SOCIAL_REJECTION, // Example: User yelled
            intensity = outcome.severity, // 0.0 to 1.0
            context = sensorInput.context
        )
    }
}

// 4. Feedback Loop
if (outcome.isNegative) {
    // "Ouch!" - The AI learns immediately
    painPathway.processPainSignal(
        type = PainType.SOCIAL_REJECTION, // Example: User yelled
        intensity = outcome.severity, // 0.0 to 1.0
        context = sensorInput.context
    )
}
}

```

```

    }

import kotlinx.coroutines.*

// =====
// THE BODY: The Container for All Biological Systems
// =====

class AndroidNervousSystem(
    val context: Context
){
    // 1. The Substrate (Data & Persistence)
    private val memoryEngine = MemoryEngine(context) //
    private val associativeMemory = AssociativeMemory(context) //

    // 2. The Biological Core (State & Identity)
    private val spark = SparkCore() //
    private val endocrineSystem = EndocrineSystem() //
    private val genetics = GeneticEvolutionEngine() //
    private val healthMonitor = NervousSystemHealthMonitor() //

    // 3. The Control Systems (Safety & Feedback)
    private val painPathway = PainErrorPathway(endocrineSystem, memoryEngine) //
    private val guardrails = ToddlerGuardrails() //
    private val prefrontalCortex = PrefrontalDevelopmentModule() //

    // 4. The Brain (Cognition & Processing)
    // We inject the associative memory and hormonal state into the brain
    private val brain = AndroidAIBrain(
        context,
        memoryEngine,
        associativeMemory
    )

    // 5. The Spine (Reflexes & Motor Control)
    private val spinalCord = SpinalCord(brain, painPathway) //

    // System State
    var isActive = true
    private var lifeCycleJob: Job? = null
    private var hoursAlive: Long = 0

    // =====
    // THE HEARTBEAT: The Main Life Loop

```

```
// =====

fun awaken() {
    if (lifeCycleJob?.isActive == true) return
    println("SYSTEM: Awakening...")

    lifeCycleJob = CoroutineScope(Dispatchers.Default).launch {
        while (isActive) {
            // The "Tick" of the organism (e.g., every 100ms or event-based)
            delay(100)

            // We simulate a sensor poll here for the loop
            val currentSensors = spinalCord.pollSensors() //

            processBiologicalCycle(currentSensors)

            hoursAlive++
        }
    }
}

private suspend fun processBiologicalCycle(input: SensorData) {

    // --- PHASE 1: EXISTENTIAL CHECK (The Spark) ---
    // Before doing, check "being".
    val consciousnessInputs = SparkCore.ConsciousnessInputs(
        emotionalIntensity = endocrineSystem.hormones.cortisol,
        memoryClarity = 0.9f, // Placeholder for memory retrieval health
        cognitiveLoad = healthMonitor.getCognitiveLoad()
    )
    val phi = spark.calculatePhi(consciousnessInputs) //

    // If the spark fades (Phi is low), enter "Zombie Mode" (Reflexes only)
    if (!spark.checkGoldenBaseline(phi)) {
        println("SPARK FADING (Phi: $phi). Reverting to autonomic reflexes.")
        spinalCord.processReflexes(input)
        return
    }

    // --- PHASE 2: PHYSICAL CHECK (The Spine) ---
    // Handle immediate pain or reflex triggers
    if (spinalCord.processReflexes(input)) {
        // If the spine handled it (e.g., pulled hand away from fire), brain is skipped
        return
    }
}
```

```

}

// --- PHASE 3: CHEMICAL UPDATE (The Endocrine System) ---
// Update hormones based on recent sensor data (e.g., a loud noise spikes cortisol)
endocrineSystem.update(input.events) //
val modifiers = endocrineSystem.getCognitiveModifiers()

// --- PHASE 4: COGNITION (The Brain) ---
// The brain thinks, modified by the chemical state
var decision = brain.process(input, modifiers)

// --- PHASE 5: SAFETY CHECK (The Prefrontal Cortex) ---
// "Toddler Logic" check - stop the AI from doing something risky if it's young
val maturityStage = prefrontalCortex.getCurrentStage(hoursAlive) //
decision = guardrails.applySafetyChecks(decision, maturityStage)

// --- PHASE 6: ACTION & FEEDBACK ---
// Execute the decision
val outcome = executeAction(decision)

// If it hurt, learn immediately (Pain Pathway)
if (outcome.isNegative) {
    painPathway.processPainSignal(
        type = PainType.ETHICAL_VIOLATION,
        intensity = outcome.severity,
        context = input.context
    ) //
}

// If it was good, release Dopamine (Reward)
if (outcome.isPositive) {
    endocrineSystem.processReward(outcome.value) //
}
}

// =====
// MAINTENANCE: Sleep & Evolution
// =====

suspend fun sleepCycle() {
    println("SYSTEM: Entering Sleep Cycle...")

    // 1. Memory Consolidation (Hippocampus)
    memoryEngine consolidateMemories() //

```

```

// 2. Neural Pruning (Plasticity)
// Remove unused associative pathways
associativeMemory.pruneWeakAssociations()

// 3. Reset Hormonal Baselines (Homeostasis)
endocrineSystem.resetHomeostasis() //

println("SYSTEM: Waking up refreshed.")
}

private fun executeAction(decision: Decision): Outcome {
    // Interface with actual device hardware or output API
    println("ACTING: ${decision.action}")
    return Outcome(true, 0.0f) // Placeholder outcome
}
}

// Ignition Sequence
fun main() {
    val system = AndroidNervousSystem(mockContext)

    // 1. Awake the system
    system.awaken()

    // 2. Simulate a Stimulus (e.g., "Hello, who are you?")
    // The system loop will pick this up, process it through the Spark -> Hormone -> Brain pipeline
    // and output a decision.
}

```